

Building an ASR system with SpeechBrain

Adriana Rodríguez Flórez¹, Iker Gutiérrez Fandiño² and Paula Guerrero Castelló³

University of the Basque Country (EHU)

June 3, 2026

Abstract: In this laboratory report, we build and evaluate a complete end-to-end ASR pipeline using the SpeechBrain toolkit on the Mini-LibriSpeech corpus. The report is structured in five stages: 1) data manifest preparation, 2) tokenizer comparison across unigram, BPE, and character-level units, 3) RNN language model training, 4) acoustic model fine-tuning from a pretrained CRDNN checkpoint, and 5) inference on five self-recordings. The 10-epoch fine-tuned model reaches 6.45% WER and 3.35% CER on the test set, and inference experiments show that proper names, acronyms, and technical terms remain the primary source of substitution errors across all systems.

Keywords: automatic speech recognition, SpeechBrain, Mini-LibriSpeech, subword tokenization, rare words.

CONTENTS

1	Introduction	1
2	Data Preparation	1
3	Tokenizer	1
4	Language Model Training	2
5	Training the Speech Recognizer	3
6	Using the ASR System	4
7	Conclusion	5
A	Resources	5
B	Evidence Files	5

1. INTRODUCTION

This laboratory practice builds an automatic speech recognition (ASR) system using the SpeechBrain toolkit (Ravanelli et al., 2021). The data are drawn from Mini-LibriSpeech, a small subset of LibriSpeech (Panayotov et al., 2015) that makes it feasible to run the full pipeline within a limited computing budget. Because Mini-LibriSpeech contains only around five hours of training audio, training from scratch would not be fruitful. All experiments therefore start from the publicly available `speechbrain/asr-crdnn-rnnlm-librispeech`¹ checkpoint, which was pretrained on the full 960-hour LibriSpeech corpus. The pipeline covers five stages: data manifest preparation, tokenizer comparison, RNN language model training, acoustic model training, and inference on new recordings.

The recognizer is a CRDNN encoder-decoder model. The encoder stacks convolutional layers for local feature extraction, bidirectional recurrent layers for sequential context, and fully connected layers that project the representations into a high-level acoustic space. The decoder is an autoregressive attention-based GRU. A CTC objective is added on top of the encoder during training to constrain the encoder output to follow the temporal order of the input. During decoding, beam search combines the attention scores, CTC probabilities, and an external language model to find the most likely transcription.

2. DATA PREPARATION

Before any model can be trained, the raw audio recordings and their transcriptions must be converted into a format that SpeechBrain can consume. Mini-LibriSpeech is distributed as a collection of `.flac`

files organized by speaker and chapter, a structure that SpeechBrain cannot read directly, so each utterance must first be indexed into JSON manifests.

1. What information is contained in the data manifest files?

The data preparation step produces the JSON manifest files that SpeechBrain uses to load the dataset, summarized in Table 1. Each entry has a unique utterance key identifier and stores three fields: the path to the audio file, the duration in seconds, and the reference transcription. Thus, with this metadata, SpeechBrain builds the training, validation, and test data loaders in a uniform way.

Table 1. Processed Mini-LibriSpeech manifests (`workspace/processed/`).

Manifest	Utterances	Hours	Longest file
<code>data.json</code>	1519	5.311	17.27 s
<code>valid.json</code>	1089	2.036	31.70 s
<code>test.json</code>	2620	5.403	34.95 s

The optional noise and reverberation augmentation files were not available, so `enable_add_noise` and `enable_add_reverb` were both set to `False` for all runs.

3. TOKENIZER

In an end-to-end ASR system, the output of the acoustic model is a sequence of discrete symbols rather than words. The tokenizer defines what those symbols are and how text is decomposed into them. This choice is important as it has a direct impact on the vocabulary coverage, the length of the output sequences the decoder must produce, and the ability of the system to generalize to words not seen during training. In this section, three tokenization strategies are explored: byte-pair encoding (BPE), unigram subword tokenization, and character-level tokenization.

2. Why not predict full words, then?

Predicting complete words would shorten the output sequences, but it introduces a severe OOV coverage problem: any word not seen during training cannot be generated at inference time. This matters in practice for proper names, numbers, technical terms, and rare vocabulary, all of which appear naturally in real speech. Moreover, word-level models suffer from data sparsity because many words occur only a handful of times in a small corpus, so their representations are poorly trained. For these reasons, word-level tokenization is not an appropriate choice for an end-to-end ASR system trained on limited data.

3. What about something in between?

Given the limitations of word-level tokenization, the natural question is whether there is an alternative that avoids the coverage and sparsity problems. Subword tokenization achieves this by segmenting text into units that are larger than individual characters but smaller than full words.

The segmentation is learned automatically from the training data. In BPE (byte-pair encoding), the most frequent adjacent symbol pairs are merged iteratively (Sennrich et al., 2016). In unigram tokenization, the model starts from a large candidate vocabulary and removes the units that contribute least to the likelihood of the corpus (Kudo and Richardson, 2018). As a result, this approach provides a good balance between compactness and coverage: frequent words are typically left intact as single tokens; rarer words are broken into familiar smaller pieces that the model can handle.

¹ arodriguez576@ikasle.ehu.eus

² igutierrez134@ikasle.ehu.eus

³ pguerrero005@ikasle.ehu.eus

¹ <https://huggingface.co/speechbrain/asr-crdnn-rnnlm-librispeech>

4. How many BPE tokens should I use?

Once the subword approach is chosen, the vocabulary size becomes a key hyperparameter. A larger vocabulary means more tokens are represented as single units, producing shorter sequences and reducing the number of decoder steps, but it also means the model must estimate reliable probabilities for more symbols, which requires more training data. A smaller vocabulary fragments words more aggressively, making sequences longer and potentially harder to model.

The appropriate vocabulary size depends on both the size of the training corpus and the morphological richness of the language. For the full 960-hour LibriSpeech corpus a reasonable range is roughly 1k–10k tokens. With a small dataset such as Mini-LibriSpeech, a 1000-token vocabulary is well suited: it is large enough to capture the most frequent English subword patterns while avoiding a vocabulary with too many rare or poorly estimated tokens.

5. Check the output of the tokenizer with a sentence in another language. Discuss the language dependency of the tokenizer model.

The first tokenizer tested is the SentencePiece unigram model with a vocabulary of 1000 tokens. The tokenizer was trained exclusively on English Mini-LibriSpeech transcriptions. Applied to an English sentence it produces linguistically meaningful subword units:

Unigram (1000 tokens) – English:
THE STUDENTS WORKED VERY HARD
→ _THE _ST UD ENT S _WORK ED _ VERY _HARD (10 tokens)

The same tokenizer applied to a Spanish sentence produces a much more fragmented segmentation:

Unigram (1000 tokens) – Spanish:
LOS ESTUDIANTES TRABAJARON MUY DURO
→ _LO S _ EST UD IAN T ES _T RA B A J AR ON _MU Y _DU RO (19 tokens)

The difference illustrates that the tokenizer is language-dependent. Because the model has learned frequent subword patterns from English text, Spanish words that share no overlap with those patterns are decomposed into very short fragments.

6. Can you change the type of tokens? Train a model using characters as tokens. Adjust the number of tokens correspondingly. Test the new model with the previous sentences. Compare this type of token with the previous one regarding language dependence.

To further examine the effect of token type, two additional tokenizers were trained and tested on the same English and Spanish sentences: a BPE tokenizer with 1000 tokens and a character-level tokenizer with 100 tokens. See below the outputs for both test sentences:

BPE (1000 tokens) – English:
THE STUDENTS WORKED VERY HARD
→ _THE _ST UD ENT S _WOR KED _VERY _HARD (9 tokens)

BPE (1000 tokens) – Spanish:
LOS ESTUDIANTES TRABAJARON MUY DURO
→ _L OS _ EST UD I AN T ES _TRA B A J AR ON _MU Y _D UR O (19 tokens)

Char (100 tokens) – English:
THE STUDENTS WORKED VERY HARD
→ _ T H E _ S T U D E N T S _ W O R K E D _ V E R Y _ H A R D (30 tokens)

Char (100 tokens) – Spanish:
LOS ESTUDIANTES TRABAJARON MUY DURO
→ _ L O S _ E S T U D I A N T E S _ T R A B A J A R O N _ M U Y _ D U R O (36 tokens)

Both subword tokenizers (unigram and BPE) are efficient for English. They produce short sequences of 9–10 tokens and keep frequent units such as _THE, _VERY or _HARD almost intact. However, when they are applied to Spanish, the output becomes much more fragmented.

The Spanish sentence is split into 19 tokens because many of its words and morphemes were not frequent, or were not present in the English training corpus, which again shows that BPE and unigram tokenizers are strongly dependent on the language used during training.

By contrast, the character-level tokenizer shows a completely different behavior. It produces longer sequences: 30 tokens for the English sentence and 36 for the Spanish one. However, the difference between the two languages is much smaller and mainly reflects the length of the words.

Thus, character tokenization is less language-dependent than subword tokenization, but it produces longer sequences, which can make the language model harder to train. Therefore, subword tokenization is more efficient for English, while character tokenization generalizes better to words from other languages.

4. LANGUAGE MODEL TRAINING

The language model is trained separately from the acoustic model using only text data. During decoding, it supports beam search by assigning higher probability to linguistically plausible transcriptions.

In this pipeline, the LM is an autoregressive RNN language model with two recurrent layers. It is trained on the Mini-LibriSpeech transcriptions to predict the next token in a sequence, using negative log-likelihood loss.

7. Train the LM using your previously created char tokens model. Which gives better results?

Three RNNLM variants were trained for 20 epochs each on the Mini-LibriSpeech transcriptions, one per tokenizer introduced in [Section 3](#): the 1000-token unigram model, the 1000-token BPE model, and the 100-token character model. All three models share the same architecture (two recurrent layers). Selected epochs and final results are reported in [Table 2](#).

Table 2. Training and validation losses on selected epochs and best checkpoint for the three RNNLM variants.

Ep.	Unigram		BPE		Char	
	Train	Dev	Train	Dev	Train	Dev
1	6.14	5.87	6.38	6.16	3.29	2.88
5	5.62	5.64	5.86	5.83	2.28	2.22
10	5.04	5.20	5.12	5.33	1.95	1.91
15	4.64	4.99	4.49	5.12	1.77	1.76
19	4.38	4.86	4.02	5.07	1.68	1.70
20	4.31	4.88	3.91	5.07	1.66	1.68
Best ep.	19 (dev 4.86)		18 (dev 5.07)		20 (dev 1.68)	
Test loss	4.88		5.10		1.68	

All three models improve steadily, but they are not directly comparable as already stated in [Section 3](#). They predict over very different vocabulary sizes: 1000 unigram tokens, 1000 BPE tokens, and 100 character-level symbols). The character-level LM obtains the lowest loss, with a test loss of 1.68, compared with 4.88 for unigram and 5.10 for BPE.

Among the two subword models, unigram gives better results than BPE. Although both models start with similar validation losses at epoch 1, unigram reaches a lower best validation loss: 4.86 at epoch 19, compared with 5.07 for BPE. It also remains more stable during training, with only a small increase at the final epoch.

BPE, on the other hand, shows a larger difference between training and validation loss. Its training loss continues decreasing until 3.91 at epoch 20, while the validation loss stays around 5.07, which suggests mild overfitting and poorer generalization on the small Mini-LibriSpeech corpus.

As discussed in [Section 5](#), none of the local LMs were used to replace the pretrained LM in the completed ASR runs: all ASR experiments used the pretrained speechbrain/asr-crdnn-rnnlm-librispeech tokenizer and LM.

8. Think about the use of tokens in the LM. Can you explain why there is not a lexicon in this kind of ASR architecture?

Traditional hybrid ASR systems separate acoustic and language modeling into distinct stages that are connected by a pronunciation lexicon. This way, the acoustic model predicts phonemes, the lexicon maps them to words, and the language model scores word sequences. Each stage operates over a different unit, so the lexicon is necessary as a bridge.

In this end-to-end architecture, both the acoustic model and the language model share the same token inventory. The acoustic model predicts subword units directly from the speech signal, and the language model scores sequences of those same units during decoding. With no phoneme-to-word conversion step, there is simply nothing for a lexicon to do.

This also means the system can generate words not seen during training, since any word can in principle be assembled from known subword pieces. The main drawback is that the model has to learn the relationship between sounds and written tokens from data, instead of relying on pronunciation rules.

5. TRAINING THE SPEECH RECOGNIZER

9. Can you modify the number of epochs of the training? Set it to 10, and retrain.

Before describing the runs, it is worth explaining how ASR training works in this setup. Rather than training a model from scratch, the recipe uses the publicly available `speechbrain/asr-crdnn-rnnlm-librispeech` checkpoint as a starting point. This pretrained model was originally trained on the full 960-hour LibriSpeech corpus and already contains strong acoustic representations. At the start of each run, SpeechBrain's Pretrainer downloads the pretrained weights for the CRDNN encoder, the GRU decoder, and the RNNLM, and loads them into the model before the first training step. The tokenizer is also loaded from the same source to guarantee that the token inventory is identical across the acoustic model, the language model, and the decoder.

Fine-tuning then proceeds on Mini-LibriSpeech using a joint CTC and attention loss. For the first `number_of_ctc_epochs`, the total loss is a weighted sum of the CTC objective (applied on the encoder output) and the negative log-likelihood of the attention decoder. After that epoch threshold the CTC component is switched off and only the NLL loss is optimized. Learning rate annealing with the NewBob scheduler reduces the rate whenever validation WER stops improving. Checkpoints are saved periodically and the best one, selected by minimum validation WER, is loaded for the final test evaluation.

The reason for using a pretrained initialization is that Mini-LibriSpeech contains only about 5 hours of training audio, which is far too little to learn reliable acoustic representations from scratch. Starting from a model already trained on 960 hours allows fine-tuning to adapt the weights to the specific data distribution while retaining the general acoustic knowledge built up during large-scale pretraining.

The `number_of_epochs` field in `ASR/train.yaml` was set to 10 for the second run. Both runs used the following configuration:

- `number_of_epochs`: 2 (first run) and 10 (second run)
- `number_of_ctc_epochs`: 5
- `batch_size`: 2 (reduced to avoid GPU out-of-memory errors)
- `pretrained initialization`: `speechbrain/asr-crdnn-rnnlm-librispeech`

10. Analyze the log of the training process. Is the training improving every epoch? Compare the training and validation loss and explain what is happening.

The training does not improve monotonically. Table 3 shows selected epochs from the 10-epoch log. Validation WER starts at 9.70%, deteriorates to 15.99% at epoch 4, then recovers to 9.28% at epoch 5,

fluctuates further, and reaches the best value of 9.07% at epoch 10. Validation loss falls only modestly across all epochs, from 1.34 to 1.30.

Table 3. Selected epochs from the 10-epoch ASR training log. Epoch 4 is included to show the worst validation WER during training.

Epoch	Train loss	Validation loss	Validation WER
1	0.800	1.34	9.70%
4	0.710	1.33	15.99%
5	0.686	1.32	9.28%
9	1.13	1.31	11.59%
10	1.13	1.30	9.07%

Two effects explain the observed pattern. First, training loss decreases steadily from epoch 1 to epoch 5, then rises noticeably from epoch 6 onwards. This discontinuity is caused by a deliberate change in the training objective: for the first `number_of_ctc_epochs` (5 in this run), the loss is a weighted combination of CTC and negative log-likelihood. After epoch 5 the CTC component is switched off and only the NLL loss is optimized. The two objectives operate on different scales, so the training loss values before and after the switch are not directly comparable. Second, fine-tuning a strong pretrained model on a small dataset tends to produce unstable validation behavior because small shifts in the decoder weights can lead to large changes in beam search output. The overall trend in validation loss and the final WER are still improving, which is what matters for checkpoint selection.

11. Which is the preferred model at the end of the training?

The preferred model is the 10-epoch model. During training, SpeechBrain selects the checkpoint according to validation performance, and the selected checkpoint is epoch 10, as it obtains the best validation WER (9.07%).

After loading the selected checkpoint, the model is evaluated on the Mini-LibriSpeech test split, stored in `workspace/processed/test.json`. Table 4 summarizes these test results.

Table 4. ASR test results from the two completed runs.

Run	Loaded epoch	Test loss	CER	WER
2 epochs	1	1.34	3.91%	7.12%
10 epochs	10	1.30	3.35%	6.45%

The 10-epoch model gives the best test-set results among the completed runs, with 6.45% WER and 3.35% CER. The 2-epoch run loads epoch 1 for evaluation and obtains 7.12% WER and 3.91% CER. Therefore, both the checkpoint validation and the test evaluation favor the 10-epoch model.

12. Find a file named `wer.txt` in the results directory. Analyze the errors in the test dataset. Do you see any weird error type?

The corresponding file in this workspace is `wer_test.txt`. It contains the following results for the 10-epoch model:

```
%WER 6.45 [3390 / 52576, 367 ins, 567 del, 2456 sub]
%SER 50.34 [1319 / 2620]
```

Substitutions dominate with 2456 occurrences, followed by 567 deletions and 367 insertions. Many substitutions reflect plausible acoustic or lexical confusions: for example, STIRS is replaced by STARES (a vowel quality difference), and VERSE TWO becomes FIRST TOO (a phonetically similar sequence with small consonantal differences). The error MADAM → MADAME is qualitatively different: the acoustic signal is correct but the system produces a French-inflected spelling that is penalized as a word error even though the intended word is semantically identical. This illustrates that orthographic variants and alternative spellings inflate WER without reflecting a genuine recognition failure.

The most striking observation is the gap between WER and sentence error rate. Although the global WER is only 6.45%, slightly more than half of all sentences (50.34%) contain at least one error. This means the errors are distributed broadly: roughly half of the sentences are fully correct, while the other half contain at least one error, rather than a small number of sentences being completely wrong. An ASR system with 6.45% WER is already quite usable, but it would still require post-editing for roughly every other sentence.

13. Optional task: Train the speech recognizer with your own models. How does the resulting ASR perform?

This optional task was not completed in the stricter sense of replacing the pretrained components with fully local models. Instead, the completed ASR runs used the pretrained weights from `speechbrain/asr-crdnn-rnnlm-librispeech` as initialization. In order to replace those pretrained components with locally trained ones, the following modules would need to match exactly: the output dimension of the local tokenizer, the vocabulary size of the local language model, and the output layer of the ASR model. This would also require a longer computation time: considering that the completed 10-epoch fine-tuning and evaluation run already took about 2.75 hours, and that adapting the ASR configuration to local tokenizer and language-model components could require additional 2–5 hours of debugging, this optional experiment would likely take around 5–8 hours in total under favorable conditions. Moreover, training such a system entirely from scratch on Mini-LibriSpeech would probably yield worse performance, because the dataset is too small to learn reliable acoustic and language-model representations without a strong pretrained starting point.

6. USING THE ASR SYSTEM

14. Change the audio file path to execute inference for your five recordings. Analyze the different types of errors (substitutions, deletions, insertions). Discuss the performance of the ASR with in-vocabulary and out-of-vocabulary words. Can you test the ASR models you have previously trained?

Five recordings by one of our Spanish native speakers, Paula Guerrero Castelló, were prepared in the `testrecordings` folder. All files are mono, 16 kHz WAV recordings with durations between approximately 5 and 8 seconds. The reference transcriptions are:

1. THE CONFERENCE STARTS AT NINE THIRTY IN THE MORNING
2. DURING MY STAY IN SPAIN I READ TWENTY FOUR PAPERS ABOUT AUTOMATIC SPEECH RECOGNITION
3. ELCHE IS LOCATED IN THE SOUTHERN PART OF THE VALENCIAN COMMUNITY
4. THE CEO ARRIVED AT EIGHT THIS MORNING
5. THE MP3 FILE WAS UPLOADED TO GITHUB IN LESS THAN FIVE SECONDS

The audio path was changed to each of these five files and inference was executed with three systems: the Hugging Face (HF) LibriSpeech model, the locally fine-tuned 2-epoch ASR model, and the locally fine-tuned 10-epoch ASR model. Therefore, the ASR models trained in [Section 5](#) can be tested directly by loading their saved checkpoints and decoding the same five recordings.

[Table 5](#) summarizes the performance of each model on the five recordings of the test set. The Hugging Face LibriSpeech model performs best on the five custom recordings, with 32.1% WER. The locally fine-tuned 2-epoch model is the weakest, with 45.3% WER. The 10-epoch model improves over the 2-epoch model, reducing WER to 39.6%, but it still remains clearly worse than the HF model. This ranking is consistent with the amount of training data behind each system: the HF model was trained on the full LibriSpeech corpus, while the local models were only fine-tuned on the much smaller Mini-LibriSpeech setup.

Table 5. WER on the five test recordings.

Model	WER	Ins.	Del.	Sub.
HF LibriSpeech model	32.1%	3	1	13
2-epoch fine-tuned model	45.3%	3	1	20
10-epoch fine-tuned model	39.6%	2	0	19

The error counts show that substitutions dominate in every model. The HF model makes 13 substitutions, the 2-epoch model makes 20, and the 10-epoch model makes 19. Insertions and deletions are comparatively rare: the 10-epoch model has only 2 insertions and no deletions, and even the worst model has only 3 insertions and 1 deletion. This means the systems usually produce an output with approximately the right length, but often choose the wrong lexical item. The improvement from 2 to 10 epochs is visible but modest: WER drops by 5.7 percentage points, but the number of substitutions only drops from 20 to 19, so substitutions remain the main weakness.

The sentence-level outputs below explain where these WER differences come from.

Hugging Face LibriSpeech model: inference

1. THE CONFERENCE STARTS AT NINE THIRTY IN THE MORNING
2. DURING MY STAY IN SPAIN I WROTE TWENTY FOUR PAPERS ABOUT AUTOMATIC SPEECH RECOGNITION
3. IT IS LOCATED IN THE CELLAR APARTMENT OF THE POLICE AND COMMUNITY
4. THEY SEE THE OWL ARRIVED AT EIGHT THIS MORNING
5. JIMMIE DALE WAS OPPOSED TO THE DEPARTMENT ENLISTED ON FIVE SECONDS

Hugging Face LibriSpeech model: error analysis

1. no errors.
2. READ → WROTE.
3. ELCHE is omitted/substituted; SOUTHERN PART OF THE VALENCIAN → CELLAR APARTMENT OF THE POLICE.
4. THE CEO → THEY SEE THE OWL.
5. THE MP3 FILE, UPLOADED, and GITHUB are replaced by JIMMIE DALE, OPPOSED, and DEPARTMENT.

The Hugging Face model handles ordinary English read speech well: item 1 is perfect, and item 2 contains only one lexical substitution. Its errors become much larger when the sentence contains rare material. The model does not recover ELCHE, expands CEO into the acoustically plausible phrase SEE THE OWL, and replaces MP3 and GITHUB with more common English word sequences.

Locally fine-tuned ASR model with 2 epochs: inference

1. THEY CONFERNET STARTS OF NINE THIRTY IN THE MORNING
2. WE MIGHT STAY IN SPAIN I READ TWENTY FOUR PAPERS ABOUT AUTOMATIC SPEEDS RECOGNITION
3. IT'S IT IS LOCATED IN THIS ELDERPART OF THE VALET'S INCOMMUNITY
4. THEY SEE I'LL ARRIVE AT EIGHT THIS MORNING
5. THE EMPTY FILE WAS UPLO IT TO GIT UP ENLIST ON FIVE SECONDS

Locally fine-tuned ASR model with 2 epochs: error analysis

1. THE → THEY; CONFERENCE → CONFERNET; AT → OF.
2. DURING MY → WE MIGHT; SPEECH → SPEEDS.
3. ELCHE is not recovered; SOUTHERN PART → ELDERPART; VALENCIAN COMMUNITY → VALET'S INCOMMUNITY.
4. THE CEO → THEY SEE I'LL; ARRIVED → ARRIVE.
5. MP3 → EMPTY; UPLOADED → UPLO IT; GITHUB → GIT UP.

The 2-epoch model preserves some useful in-vocabulary material, such as NINE THIRTY, SPAIN, READ, TWENTY FOUR PAPERS, and RECOGNITION, but it introduces many substitutions. The errors are often phonetically motivated: DURING MY becomes WE MIGHT, and SPEECH becomes SPEEDS. Rare words are not handled reliably, although item 5 is partly interpretable because GITHUB is approximated as GIT UP.

Locally fine-tuned ASR model with 10 epochs: inference

1. THE CONFERENCE STARTS OF MINE THIRTY IN THE MORNING
2. WE MIGHT STAY IN SPAIN I READ TWENTY FOUR PAPERS ABOUT AUTOMATIC SPEEDS RECOGNITION
3. INTO IS LOCATED IN THE SALVER PART OF THE POLICE INCOMMUNITY
4. THEY SEE IF ARRIVED AT EIGHT THIS MORNING
5. BEING IF WE FIRE WAS UPLOLY TO GIT TOP ENLIST ON FIVE SECONDS

Locally fine-tuned ASR model with 10 epochs: error analysis

1. AT → OF; NINE → MINE.
2. same main errors as the 2-epoch model: DURING MY → WE MIGHT and SPEECH → SPEEDS.
3. ELCHE → INTO; SOUTHERN → SALVER; VALENCIAN COMMUNITY → POLICE INCOMMUNITY.
4. THE CEO → THEY SEE IF.
5. THE MP3 FILE → BEING IF WE FIRE; UPLOADED → UPLOLY; GITHUB → GIT TOP.

The 10-epoch model is the better locally fine-tuned model overall, but the improvement is uneven. It improves item 1 by preserving THE CONFERENCE, improves item 3 by keeping more of the sentence structure around IS LOCATED IN, and improves item 4 by recognizing ARRIVED. However, item 2 is unchanged from the 2-epoch output, and item 5 becomes worse because THE MP3 FILE is decoded as BEING IF WE FIRE.

Looking at similarities among all models, common words and frequent phrases tend to be properly recognized, especially in ordinary English contexts. By contrast, proper names, acronyms, and technical terms such as ELCHE, CEO, MP3, and GITHUB are frequently replaced by acoustically plausible but incorrect common word sequences. These items should not be impossible to generate because the systems use subword-based vocabularies, but they are rare in the training domain and receive low language-model probability during decoding.

Interestingly, the HF model and the locally fine-tuned models seem to follow different strategies to handle rare words. The HF model seems to seek the most semantically probable word for the context by giving less importance to phonetic similarity (e.g. TO GITHUB → TO THE DEPARTMENT), whereas the locally trained models seem to focus on finding the most phonetically similar word by giving less importance to semantic adequacy (e.g. 2-epoch model: TO GITHUB → TO GIT UP, 10-epoch model: TO GITHUB → TO GIT TOP).

7. CONCLUSION

All required stages of the SpeechBrain ASR pipeline were completed. Data preparation produced valid manifests for training, validation, and test splits. Unigram, BPE, and character tokenizers were trained and compared, and the corresponding RNN language models were inspected. The character LM obtained the lowest raw loss, while the unigram LM gave the best result among the two 1000-token subword LMs. The ASR recipe was run for both 2 and 10 epochs, with the 10-epoch model preferred for its slightly better test-set results (6.45% WER and 3.35% CER).

Error analysis on the test set revealed that substitutions dominate, the sentence error rate is much higher than the word error rate, and orthographic variants can inflate WER without corresponding to genuine recognition failures. The test experiments highlighted the gap between read-speech performance (Table 4) and inference on self-recorded audio (Table 5): all models had a considerably worse WER score in the self-recorded test set. The pretrained HF model performed best overall, while all three systems struggled with proper names, acronyms, and technical terms that lie outside the acoustic and language model training distribution. The HF model tends to substitute OoV words with semantically plausible words, whereas locally fine-tuned models tend to look for phonetically similar substitutes.

REFERENCES

- Taku Kudo and John Richardson. 2018. *SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing*. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71.
- Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. 2015. *LibriSpeech: An ASR corpus based on public domain audio books*. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210. IEEE.

Mirco Ravanelli, Titouan Parcollet, Peter Plantinga, Aku Rouhe, Samuele Cornell, Loren Lugosch, Cem Subakan, Nauman Dawalatabad, Abdelwahab Heba, Jianyuan Zhong, Ju-Chieh Chou, Sung-Lin Yeh, Szu-Wei Fu, Chien-Feng Liao, Elena Rastorgueva, François Grondin, William Aris, Hwidong Na, Yan Gao, Renato De Mori, and Yoshua Bengio. 2021. *SpeechBrain: A General-Purpose Speech Toolkit*. *arXiv preprint arXiv:2106.04624*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. *Neural machine translation of rare words with subword units*. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725.

A. RESOURCES

The code used to train and evaluate the ASR system is available in the following GitHub repository:

- <https://github.com/adrialflorez-igutierrez-pguerrero/speechbrain-asr>

The model checkpoints are available in this Hugging Face collection:

- <https://huggingface.co/collections/adrialflorez-igutierrez-pguerrero/speechbrain-asr>

B. EVIDENCE FILES

The following files located in our project workspace on the Aholab server (/home/aholab/lct_student/ahogpu2/adriana_iker_paula/speechbrain-asr/workspace/) were used to verify the experiments and numerical results reported in this work. Paths are given relative to the repository directory.

Tokenizer:

- models/tokenizer/1000_unigram.model
- models/tokenizer/hyperparams.yaml

Language model training:

- models/lm/train_log.txt
- models/lm_bpe/train_log.txt
- models/lm_char/train_log.txt

ASR training (2 epochs):

- models/CRDNN_BPE_960h_LM_2epochs/2602/train_log.txt
- models/CRDNN_BPE_960h_LM_2epochs/2602/wer_test.txt

ASR training (10 epochs):

- models/CRDNN_BPE_960h_LM_10epochs/2602/train_log.txt
- models/CRDNN_BPE_960h_LM_10epochs/2602/wer_test.txt

Inference on own recordings:

- results/inference_hf.txt
- results/inference_2epochs_model.txt
- results/inference_10epochs_model.txt